Screening Coding-Agent Task Failures with Normal-Only Reconstruction

Tun-Chieh Yang (315553010) **Jia-Cheng Wu (314554059)** **Min-Quan Wang (315553008)**
Wei-Chia Chang (315553011) Chun-Ting Hou (315554022)
Team G04, Data Mining (Fall 2026), NYCU
https://github.com/calculusfkyou/nycu-data-mining-assignment-2-115-1

Abstract

Release engineers need to identify unreliable coding-agent patches before integration. We study task-end failure screening on 284 public SWE-agent trajectories from SWE-bench Lite, representing execution logs and patch structure with 24 features. An autoencoder fitted only on 32 successful training runs achieves test average precision (AP) 0.978 ± 0.004 and $F_2 = 0.948 \pm 0.006$ over five initialization seeds. Its AP gap to supervised XGBoost is not significant under a paired test ($p = 0.449$). Controlled feature masking improves AP over patch-only inputs by 0.0143, while contamination of three training rows lowers AP by 0.0099. These gains require caution: failures constitute 92.6% of the test set, and the autoencoder falsely flags 75% of its 12 successes. We analyze three errors, reconstruct a flagged sample's feature contributions, and show why high AP alone does not establish a useful release gate.

1 Introduction

Coding agents edit repositories through commands, editor actions, and test executions (Yang et al., 2024). A finished execution need not resolve its assigned issue. Our stakeholder is a release engineer deciding which completed agent patches require additional inspection before integration. A high score requests human review; a low score never replaces tests or authorizes deployment.

Our application belongs to **Category 4: Text / Logs**: irregular command and observation sequences have heterogeneous vocabulary, repository conventions, and error semantics. We aggregate these into tabular features so that a small normal-only model can be inspected. The question is: *Do workflow features help identify unresolved tasks beyond final patch structure, and does a normal-only detector offer a useful alert tradeoff?*

The detection unit is one *completed* trajectory and its final patch, not a prefix. Normal means resolved according to the official execution results; anomalous means unresolved, including budget termination. For example, the successful `scikit-learn-14894` run takes 25 steps despite 19 error-like observations, whereas unresolved `sympy-15345` takes 11 steps with zero such observations. These counterexamples motivate multivariate screening instead of treating an error string as proof of failure. The label does not establish deceptive intent or an explicit false completion claim.

Our contributions are (1) a reproducible shared-partition, three-tier comparison; (2) controlled contamination, feature-mask, and threshold experiments; and (3) traceable error, attribution, and operational analyses that expose the limits of a high-AP detector. Sections 7–8 quantify each contribution.

2 Related Work

SWE-bench (Jimenez et al., 2024) collects 2,294 real GitHub issues from 12 Python repositories and evaluates language-model-generated patches using execution tests. Its original best model resolves only 1.96% of issues. This is supervised *evaluation* of issue resolution, not training a failure monitor. SWE-agent (Yang et al., 2024) instead studies interactive, tool-using language-model agents: its agent-computer interface achieves 12.5% pass@1 on full SWE-bench and 87.7% on HumanEvalFix. These results concern different evaluation sets and must not be compared numerically to our screening AP.

Deep SVDD (Ruff et al., 2018) learns a normal representation with a one-class hypersphere objective, demonstrating anomaly detection on MNIST, CIFAR-10, and adversarial stop signs. It provides a contrasting normality assumption: com-

pactness around a center rather than reconstruction. We choose a small autoencoder because additive per-feature reconstruction errors directly explain alerts; its flexibility can also reconstruct failures, which our ablation examines. Isolation or one-class distance models remain relevant alternatives, not tested superiority claims.

XGBoost (Chen and Guestrin, 2016) is a supervised boosted-tree system for structured features. We use its public implementation as Tier 3 to learn discriminative interactions from labeled training failures, rather than present our SWE-bench result as a result of the 2016 paper. Tier 1 is our own transparent two-feature heuristic. Unlike issue-solving benchmarks, we ask whether a completed run deserves extra review, without making additional LLM calls.

3 Task Definition and Data

Source and labels. We use the public `20240402_sweagent_gpt4` Lite evaluation run.[1] The official results list 284 generated runs and 16 with no generation; all 284 have cached trajectories. Labels are membership in the official `resolved` list, checked by instance ID. There are 54 successes (19.01%) and 230 failures (80.99%), or 1:4.26. The 190 voluntary submissions contain 140 failures; 94 budget-terminated submissions contain 90 failures. We do not infer that every failure claimed success. Benchmark tests can miss semantic errors, and non-resolution can include execution issues; these are label limitations.

Failures are *common* here, unlike classical rare-event anomaly detection. "Anomaly" means departure from a successful reference, not statistical rarity. Always predicting failure obtains 81.0% accuracy on the complete dataset; test prevalence is even higher. We report AP (often called PR-AUC, computed as average precision, not trapezoidal area), ROC-AUC, and F_1/F_2, with an always-alert comparator.

Representation. We extract 16 workflow features: step and command counts, tool diversity and entropy, longest *consecutive* repetition, error count/rate/streak, stalled observations, reproduction-script indicator, test-command count, last-observation error flag, cost, sent/received tokens, and tokens per step. Eight patch features measure characters, added/removed lines, changed files, empty patch, test-string indicator, edit ratio, and hunk count. Raw `exit_status`, resolution, and instance/repository IDs are excluded from inputs. No forced-termination indicator enters the patch-only comparison. Empty patch indicators preserve missing-output information; genuine zeros remain. Both regex indicators are noisy: error matches include expected reproduction failures or quoted code, and the test indicator matches strings anywhere in the diff, not only changed file paths. Raw hashes and source URLs appear in `Dataset/provenance.json`; selectable features and split IDs are committed.

[1] https://github.com/SWE-bench/experiments/tree/main/evaluation/lite/20240402_sweagent_gpt4

Feature	Normal (32) Mean	Normal (32) SD	Failure (50) Mean	Failure (50) SD
Steps	15.00	8.30	22.32	9.01
Error observations	5.44	5.02	8.68	7.86
Stalled turns	0.91	1.06	2.32	4.86
Cost (USD)	1.61	1.05	2.64	1.28
Tokens received	1637.53	1022.00	3021.08	1440.34
Patch characters	1503.19	2377.12	1590.24	1283.92
Added lines	35.62	145.38	24.06	28.58

Table 1: Raw feature summary on the training partition only. Counts are per run; cost is USD.

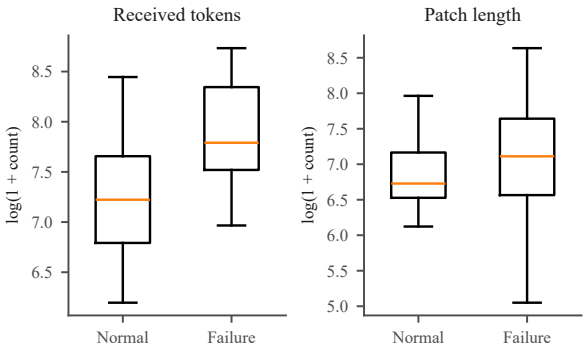


Figure 1: Training-only distributions of received-token count and patch character count, shown as $\log(1+x)$ boxplots (outliers omitted). Classes overlap, especially for patch length.

4 Exploratory Feature Analysis

Only the 82-row *training* partition is used for EDA (32 normal, 50 failures). The autoencoder later fits on its normal subset only. Figure 1 contrasts distributions, and Table 2 quantifies separation with KS and Wasserstein distance after standardizing on training rows. KS p-values are descriptive and uncorrected across features, not proof of predictive benefit.

Feature	Std. W	KS	p
tokens_received	0.957	0.505	4.3e-05
total_cost_usd	0.802	0.394	0.0031
num_steps	0.783	0.364	0.008
num_error_observations	0.464	0.240	0.17
patch_char_length	0.368	0.245	0.16

Table 2: Training separability; W is dimensionless after scaling.

Received tokens separate most strongly ($W = 0.957$, KS 0.505), followed by cost ($W = 0.802$), suggesting resource-intensive failure patterns; patch length alone is less informative. The saved Spearman matrix identifies exact redundancy between step and command counts (one action per step), as well as correlated token/cost features. We retain the fixed representation to audit that redundancy rather than select columns on test performance; this can over-weight activity volume. Standardization prevents raw token magnitudes from dominating MSE. Log transforms and deduplication are plausible next experiments, not claimed improvements.

We test three hypotheses: **H1**, workflow features improve ranking beyond patch structure; **H2**, fitting weights on failures erodes successful-reference separation; **H3**, validation F_2 calibration trades more alerts for fewer missed failures than F_1. These are exploratory hypotheses in a revised analysis, not preregistered discoveries.

5 Method

Figure 2 summarizes six checkpoints. Inputs $x \in \mathbb{R}^{24}$ are standardized using only successful training rows, $\tilde{x}_d = (x_d - \mu_d)/\sigma_d$. The 24–16–6–16–24 autoencoder uses LayerNorm and ReLU in hidden layers and a linear output, minimizing successful-training reconstruction MSE. The bottleneck limits memorization, although only 32 normal rows make overfitting plausible. LayerNorm supports small batches; a fixed compact architecture avoids a broad search on the tiny validation set.

$$s(x) = \frac{1}{24}\sum_{d=1}^{24}(\tilde{x}_d - f_\theta(\tilde{x})_d)^2. \qquad (1)$$

Scores are dimensionless standardized squared errors. We search 91 validation-score percentiles from 5 to 95, choose the first maximum of validation F_2, and alert if $s(x) \geq \tau$. Seed 42 yields $\tau = 0.1758$. Weights and scaler never use validation labels. This is semi-supervised calibration, not entirely label-free learning.

A false negative means an unresolved patch avoids extra review; a false positive wastes review on a resolved run. $F_2 = 5TP/(5TP + 4FN + FP)$ weights FN more heavily, but is *not* a dollar-optimal threshold. The illustrative \$1,200 missed-failure and \$30 review costs in Section 8 imply a different objective; our threshold experiment tests the narrower F_2 motivation.

Tier 1 scores $0.5e/3 + 0.5r/5$ from error count e and consecutive-tool streak r, alerting at 1. These transparent scale choices are fixed heuristics, not validated optimal cutoffs. Tier 3 fits supervised XGBoost on the shared mixed training partition; its failure probability is calibrated over thresholds 0.10–0.90 in steps of 0.01 using validation F_2. Its scaler is fitted only on mixed training rows. Supervision is an explicit advantage, not a guaranteed upper bound.

6 Experimental Setup

The fixed split has train 82 (32 normal/50 failure), validation 40 (10/30), and test 162 (12/150), or 28.87/14.08/57.04% of all runs. The original test membership is preserved. During this audit, 50 failures previously assigned to development validation were moved into training so Tier 3 and contamination experiments no longer fit on validation rows. Every method shares these IDs, labels, and evaluation metrics; Tier 2 fits only the 32 normal training rows. Splits are by independent issue/run, not windows of a time series. Shared repositories remain across splits, so this is not unseen-repository generalization.

The test set was inspected in an earlier draft and rerun after correctness repairs; we cannot claim a pristine once-only holdout. No new architecture or threshold objective was chosen to maximize this revised test score. Independent fresh runs are needed for confirmatory evidence.

We retain fixed engineering settings, with no hyperparameter search: AdamW learning rate 0.005, weight decay 10^{-4}, 150 epochs, batch 16; XGBoost 100 trees, depth 3, learning rate 0.05. The only search is the stated validation thresholds. Runs use an AuthenticAMD CPU, one thread, Python 3.11.9, PyTorch 2.14.1, scikit-learn 1.9.1, and XGBoost 3.2.0; the processor identifier is saved with outputs. Seeds are 42, 123, 456, 789, 2026. SD uses $n - 1$; seeds vary initialization on

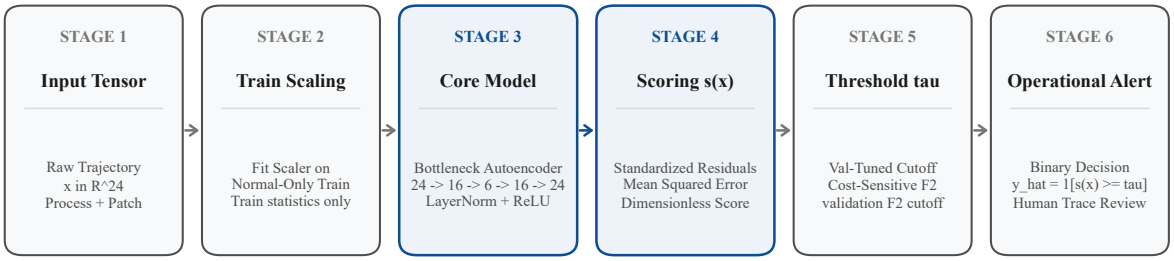


Figure 2: Tier 2: task-end feature vector, normal-training standardization, reconstruction, MSE score, validation-only calibration, and human review alert. No automatic release decision is evaluated.

one split and do not measure dataset uncertainty. XGBoost has zero seed variation with its deterministic default sampling.

AP evaluates ranking but must be compared to test failure prevalence, 150/162=0.926. ROC-AUC measures success/failure discrimination without that prior baseline. F_2 emphasizes missed failures; F_1 provides a balanced threshold summary. We report wall-clock training including initialization, plus median warmed batched scoring time (20 repeats) divided by 162. This is throughput, not single-request end-to-end latency, and excludes feature extraction and human review.

7 Results

Table 3 shows Tier 2 AP 0.978 vs. Tier 1 0.962 and Tier 3 0.980. The paired seed t-test gives $p = 0.000723$ against Tier 1 and $p = 0.449$ against Tier 3. Paired test-row bootstrap intervals (1,000 resamples, seed 42 model) for AP differences are $[0.0019, 0.0417]$ and $[-0.0147, 0.0206]$, respectively. These answer different uncertainty questions; neither proves equivalence to Tier 3. Figure 3 shows seed 42 curves, not seed means. The supervised model can learn failure interactions directly; the autoencoder only models success, explaining why it need not beat XGBoost's ranking.

High AP is partly a consequence of failure prevalence. Always alert has higher F_2 (0.984) than every fitted method; Tier 2 instead spares some reviews. Its mean confusion counts are TP 142.4, FN 7.6, FP 9, TN 3, with failure recall 0.949 and success false-alarm rate 0.750. With just 12 test successes, one case changes FPR by 8.3 percentage points. This does not validate a safe automatic gate.

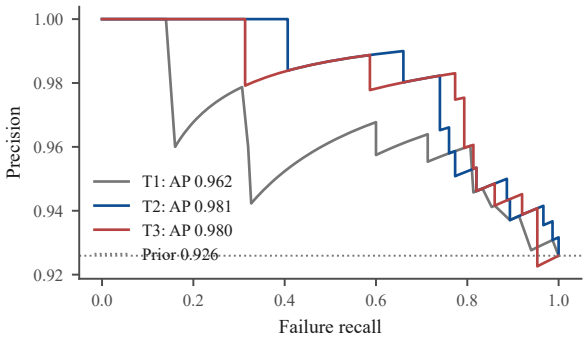


Figure 3: Seed 42 precision–recall curves; dotted line is failure prevalence. Legend AP values are computed from the plotted scores; ROC-AUC is reported in Table 3.

Controlled contamination (H2). Replace 0, 1, 2, or 3 of the 32 normal fitting rows with *training-partition* failures. Actual fractions are 0, 3.125, 6.25, 9.375%; 1% is unrepresentable at this size. Nested seeded anomaly subsets, row count, architecture, optimizer, and the clean-normal scaler are held fixed; validation is reserved for recalibration. Table 4 shows the 9.375% AP drop of 0.0099 exceeds clean seed SD 0.0039 by 2.5 times. H2 is supported in this protocol, plausibly because failures become reconstructible; it does not establish a universal contamination boundary.

Feature masks (H1). After normal-only scaling, zero inactive coordinates but retain all 24 input/output units and the same network. This isolates feature availability without changing model capacity. Full vs. patch-only AP improves by 0.0143, exceeding either seed SD (Table 5); H1 is supported for ranking. However, patch-only F_2 is *higher* by 0.006, within its SD 0.013. Thus workflow features improve ranking, not every decision metric. Process-only trails full AP by 0.0031,

Method	AP	ROC-AUC	F_1	F_2	ms/sample	Train (s)
T1: Heuristic	0.962 ± 0.000	0.691 ± 0.000	0.817 ± 0.000	0.751 ± 0.000	$\mathbf{0.0001 \pm 0.0000}$	$< \mathbf{10^{-4}}$
T2: Autoencoder	0.978 ± 0.004	0.782 ± 0.024	$\mathbf{0.945 \pm 0.004}$	$\mathbf{0.948 \pm 0.006}$	0.0018 ± 0.0000	0.6411 ± 0.8658
T3: XGBoost	$\mathbf{0.980 \pm 0.000}$	$\mathbf{0.794 \pm 0.000}$	0.940 ± 0.000	0.940 ± 0.000	0.0034 ± 0.0001	0.0130 ± 0.0006
Always alert	0.926	0.500	0.962	0.984	–	–

Table 3: Five seeds, same test IDs, mean $\pm$ sample SD. Bold: best among the three tiers, not evidence of significance. Timing is warmed batch throughput; training includes initialization. Always alert is an additional prevalence comparator, not a fitted tier.

Replaced	AP	ΔAP	F_2
0.000%	0.9783 ± 0.0039	$+0.0000$	0.948 ± 0.006
3.125%	0.9767 ± 0.0041	-0.0016	0.921 ± 0.036
6.250%	0.9707 ± 0.0027	-0.0076	0.915 ± 0.031
9.375%	0.9684 ± 0.0016	-0.0099	0.916 ± 0.013

Table 4: Weight-contamination ablation; scaler fixed to clean normals.

Active inputs	AP	ΔAP	F_2
Full	0.9783 ± 0.0039	$+0.0000$	0.948 ± 0.006
Process	0.9751 ± 0.0040	-0.0031	0.930 ± 0.023
Patch	0.9639 ± 0.0047	-0.0143	0.954 ± 0.013

Table 5: Single-factor masking, fixed 24-dimensional architecture. Full/process/patch activate 24/16/8 coordinates.

within full-model SD; complementarity is modest, not decisive.

Threshold rule (H3). Reuse identical weights and scores and change only calibration. Validation-optimal F_1 and F_2 choose identical thresholds for every seed, yielding identical mean FN 7.6 and FP 9: H3 is *not supported*. The 95th percentile of validation-normal scores yields FN 144 and FP 0, illustrating costly under-alerting when only ten validation successes calibrate a tail threshold. These outcomes appear in the saved threshold-ablation table; ranking AP is unchanged by construction.

8 Analysis

Three concrete errors. Table 6 reports the seed 42 model with $\tau = 0.1758$; $s(x) - \tau$ distinguishes a boundary miss from a low-score miss. Full IDs are `scikit-learn__scikit-learn-14894` (1), `django__django-14155` (2), and `sympy__sympy-15345` (3). All three end with a recorded `submit` action; raw final-action/diff excerpts are saved for inspection.

Case	Steps	Errors	Added	$s(x)$	$s - \tau$
1: FP	25	19	8	1.5674	+1.3916
2: FN boundary	9	0	15	0.1694	-0.0065
3: FN low score	11	0	2	0.0630	-0.1128

Table 6: Actual test errors. Steps/errors/added lines are raw counts, scores dimensionless.

Case 1 is a successful SVM sparse-matrix patch flagged despite resolution: 19 of 25 observations match our broad error regex. Expected error reproduction can therefore resemble failure; the aggregate cannot distinguish recovery from persistent defects. Case 2 modifies Django's `ResolverMatch` handling of partial functions, with 15 added lines but no matched errors. Its score is only 0.0065 below threshold, so calibration uncertainty could change the alert; that alone cannot diagnose the unresolved semantic requirement. Case 3 changes SymPy's Mathematica printer, adds two lines, and also contains no matched errors, yet fails the official evaluation with a score 0.1128 below threshold. It is *not* an empty-patch or proven premature-abort case. These errors expose a shared blind spot: syntactically quiet edits can remain semantically incomplete, while noisy successful debugging triggers alerts. First fixes should distinguish expected from unresolved errors and verify task-specific behavior on new runs; making the network larger cannot supply missing semantic evidence.

Why one sample is flagged. For `django__django-11630`, $s = 3.8673 > \tau$, and 14 stalled observations contribute 85.04% of squared reconstruction error; absence of a reproduction script contributes 5.89% (Figure 4). Feature contributions sum to 100%, and their mean squared errors recover the reported score. Repeated observations are unusual relative to successful training runs and justify opening the trace, but do not establish causation or a failed

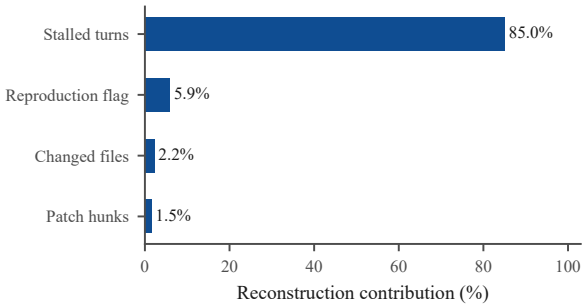


Figure 4: Seed 42 flagged failure: top four additive reconstruction contributions, not causal importance. The complete raw values are saved with the attribution table.

patch. An engineer should inspect commands and tests rather than treat the explanation as a verdict.

Operational translation. Consider an *illustrative*, not measured deployment: 500 tasks/month, 80% failure, $1,200 per missed failure, and 0.5 review hours at $60/hour per alert. Apply each seed's test recall and FPR to 400 failures and 100 successes. Tier 1 projects 114.7 escapes and 327.0 reviews; Tier 2 projects 20.3 escapes and 454.7 reviews, including 75 false alerts. Review time rises from 163.5 to 227.4 hours. Total scenario cost (escapes plus *all* alerts) drops from $147,410 to $37,962; this is a conditional model, not observed savings. Reviewing all 500 costs only $15,000 with no unreviewed failures under the idealized assumption that review catches every defect. Thus even this cost model favors universal review over our detector. At lower deployment prevalence, precision and optimal thresholds change further. A realistic next study needs measured review capacity, reviewer sensitivity, and local prevalence.

9 Conclusion and Limitations

Workflow features improve failure ranking beyond patch-only inputs by 0.0143 AP in our controlled experiment. Normal-only reconstruction reaches 0.978 AP and 0.948 F_2, with no significant AP gap to supervised XGBoost; it does not establish equivalence. Contamination reduces AP by 0.0099, while F_2 and F_1 calibration give identical decisions. High failure prevalence and 75% success FPR mean the current evidence supports trace triage research, not autonomous release approval.

The most consequential lesson is to compare a detector with an always-alert policy and count every review. Future work should first evaluate error recovery and task semantics, then collect fresh model/repository runs with adequate successful examples. Coverage is restricted to one old GPT-4 agent run, and benchmark/extraction noise and shared repositories limit transfer. Test reuse during repairs prevents confirmatory claims. We have not measured prefix detection, live latency, privacy compliance, or production outcomes. Changes to agent tools or token budgets require new calibration; retain code/log access controls, monitor local FPR and score drift, and re-evaluate on a new labeled window before changing the gate.

Contributions of Each Member

- Tun-Chieh Yang (315553010): Pending confirmation.
- Jia-Cheng Wu (314554059): Pending confirmation.
- Min-Quan Wang (315553008): Pending confirmation.
- Wei-Chia Chang (315553011): Pending confirmation.
- Chun-Ting Hou (315554022): Pending confirmation.

AI Usage Statement. Codex (GPT-6.1 Sol) audited the protocol, revised feature extraction and scripts, ran local experiments, checked references, and drafted report/slide revisions. Earlier repository materials attribute plotting and formatting assistance to Claude and Gemini; their exact versions and scope remain unconfirmed. Member contributions and team verification are pending confirmation; this draft does not attest to work that has not been confirmed.

References

Tianqi Chen and Carlos Guestrin. 2016. XGBoost: A scalable tree boosting system. In *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, pages 785–794.

Carlos E. Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik Narasimhan. 2024. SWE-bench: Can language models resolve real-world GitHub issues? In *International Conference on Learning Representations*.

Lukas Ruff, Robert Vandermeulen, Nico Goernitz, Lucas Deecke, Shoaib Ahmed Siddiqui, Alexander Binder, Emmanuel Müller, and Marius Kloft. 2018. Deep one-class classification. In *Proceedings of the 35th International Conference on Machine Learning*, volume 80, pages 4393–4402. PMLR.

John Yang, Carlos E. Jimenez, Alexander Wettig, Kilian Lieret, Shunyu Yao, Karthik Narasimhan, and Ofir Press. 2024. SWE-agent: Agent-computer interfaces enable automated software engineering. In *Advances in Neural Information Processing Systems*, volume 37.